

7. Write a Swings program to a) mouse and keyboard events

a. Swing Mouse and Keyboard Events			
Type	Input	Expected Output	Explanation
Success	Mouse clicked on frame	Mouse clicked message displayed	Mouse event
Success	Mouse moved	Coordinates displayed	Mouse motion event
Success	Key pressed A	Key pressed message displayed	Keyboard event
Failure/Edge	Press special key like Shift	Key event handled	Non-character key
Failure/Edge	Multiple rapid key presses	Events handled correctly	Event handling robustness

Algorithm:

1. **Start** the program.
2. Create a class MouseKeyboardEvents that extends JFrame and implements:
 - MouseListener
 - MouseMotionListener
 - KeyListener
3. Create a JLabel named statusLabel to display mouse and keyboard event messages.
4. Set the frame title, size, layout, and close operation.
5. Add the label to the center of the frame.
6. Register the frame as a listener for:
 - Mouse events using addMouseListener()
 - Mouse motion events using addMouseMotionListener()
 - Keyboard events using addKeyListener()
7. Make the frame focusable so that it can receive keyboard events.
8. Display the frame.
9. **Handle mouse events:**
 - When the mouse is clicked, display its X and Y coordinates.
 - When the mouse enters the frame, display "Mouse entered frame".
 - When the mouse exits the frame, display "Mouse exited frame".
10. **Handle mouse motion:**
 - When the mouse is moved, display its current X and Y coordinates.
11. **Handle keyboard events:**
 - When a key is pressed, display the key name and key code.
 - When a key is released, display the key name.
 - When a key is typed, display the character if it is not a control character.
12. Update statusLabel whenever an event occurs.
13. Run the GUI creation using SwingUtilities.invokeLater().
14. **Stop** the program when the user closes the window.

Program:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class MouseKeyboardEvents extends JFrame
    implements MouseListener, MouseMotionListener, KeyListener {

    private JLabel statusLabel;

    public MouseKeyboardEvents() {
        setTitle("Mouse and Keyboard Events Demo");
        setSize(500, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        statusLabel = new JLabel("Perform mouse or keyboard actions",
SwingConstants.CENTER);
        statusLabel.setFont(new Font("Arial", Font.BOLD, 16));
        add(statusLabel, BorderLayout.CENTER);

        addMouseListener(this);
        addMouseMotionListener(this);
        addKeyListener(this);

        setFocusable(true);
        setVisible(true);
    }

    // Mouse Events
    @Override
    public void mouseClicked(MouseEvent e) {
        statusLabel.setText("Mouse clicked at (" + e.getX() + ", " + e.getY() + ")");
    }

    @Override
    public void mousePressed(MouseEvent e) {}

    @Override
    public void mouseReleased(MouseEvent e) {}

    @Override
    public void mouseEntered(MouseEvent e) {
        statusLabel.setText("Mouse entered frame");
    }
}
```

```

@Override
public void mouseExited(MouseEvent e) {
    statusLabel.setText("Mouse exited frame");
}

// Mouse Motion Event
@Override
public void mouseMoved(MouseEvent e) {
    statusLabel.setText("Mouse moved: X=" + e.getX() + ", Y=" + e.getY());
}

@Override
public void mouseDragged(MouseEvent e) {}

// Keyboard Events
@Override
public void keyPressed(KeyEvent e) {
    statusLabel.setText(
        "Key Pressed: " +
        KeyEvent.getKeyText(e.getKeyCode()) +
        " (Code: " + e.getKeyCode() + ")"
    );
}

@Override
public void keyReleased(KeyEvent e) {
    statusLabel.setText(
        "Key Released: " +
        KeyEvent.getKeyText(e.getKeyCode())
    );
}

@Override
public void keyTyped(KeyEvent e) {
    if (!Character.isISOControl(e.getKeyChar())) {
        statusLabel.setText("Key Typed: " + e.getKeyChar());
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new MouseKeyboardEvents());
}
}

```

b) factorial computation.

b. Swing Factorial Computation			
Type	Input	Expected Output	Explanation
Success	5	120	Normal factorial
Success	0	1	Zero factorial
Success	6	720	Standard case
Failure/Edge	-3	Error message	Negative factorial invalid
Failure/Edge	abc	Error message	Non-numeric input

Algorithm:

1. **Start** the program.
2. Create a class FactorialSwing that extends JFrame and implements ActionListener.
3. Create the required GUI components:
 - JLabel for the input message.
 - JTextField to accept the number.
 - JButton to calculate the factorial.
 - JLabel to display the result.
4. Set the frame title, size, close operation, and layout.
5. Add the input label, text field, button, and result label to the frame.
6. Register the button with an ActionListener so that an action is performed when the button is clicked.
7. Display the GUI window.
8. When the **Calculate Factorial** button is clicked:
 - Read the value entered in the text field.
 - Convert the entered string into an integer using Integer.parseInt().
9. Check whether the entered number is negative.
 - If $n < 0$, display an error message because factorial is not defined for negative numbers.
10. If the number is non-negative:
 - Initialize factorial = 1.
 - Repeat from $i = 1$ to n .
 - Multiply factorial by i in each iteration.
11. Display the calculated factorial in the result label.
12. If the user enters an invalid value, catch the NumberFormatException and display an error message.
13. **Stop** the program when the user closes the window.

Program:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
```

```
public class FactorialSwing extends JFrame implements ActionListener {
```

```
    JLabel lblInput, lblResult;
```

```
JTextField txtNumber;  
JButton btnCalculate;
```

```
public FactorialSwing() {  
    setTitle("Factorial Calculator");  
    setSize(400, 200);  
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    setLayout(new GridLayout(3, 2, 10, 10));  
  
    lblInput = new JLabel("Enter a Number:");  
    txtNumber = new JTextField();  
  
    btnCalculate = new JButton("Calculate Factorial");  
    btnCalculate.addActionListener(this);  
  
    lblResult = new JLabel("Result will be displayed here");  
  
    add(lblInput);  
    add(txtNumber);  
    add(btnCalculate);  
    add(new JLabel(""));  
    add(lblResult);  
  
    setVisible(true);  
}
```

```
@Override  
public void actionPerformed(ActionEvent e) {  
    try {  
        int n = Integer.parseInt(txtNumber.getText());  
  
        if (n < 0) {  
            lblResult.setText("Error: Factorial not defined for negative numbers");  
            return;  
        }  
  
        long factorial = 1;  
  
        for (int i = 1; i <= n; i++) {  
            factorial *= i;  
        }  
  
        lblResult.setText("Factorial of " + n + " = " + factorial);  
  
    } catch (NumberFormatException ex) {  
        lblResult.setText("Error: Please enter a valid numeric value");  
    }  
}
```

```
    }  
}  
  
public static void main(String[] args) {  
    new FactorialSwing();  
}  
}
```